GDS(Graph Data Science)란?

- **GDS**는 Neo4j에서 제공하는 **그래프 데이터 과학 라이브러리**입니다.
- 그래프 구조를 분석하고 머신러닝 알고리즘을 적용할 수 있는 강력한 도구입니다.
- Python에서 `graphdatascience` 라이브러리를 통해 사용할 수 있습니다.

GDS의 주요 기능

기능	설명
그래프 알고리즘	PageRank, Centrality, Community Detection 등
그래프 임베딩	노드를 벡터로 변환하여 ML 모델에 활용
그래프 네이티브 ML	그래프 구조를 직접 활용한 머신러닝
성능 최적화	대규모 그래프 데이터를 효율적으로 처리

주요 알고리즘 카테고리

카테고리	알고리즘 예시	활용 분야
중앙성(Centrality)	PageRank, Betweenness, Closeness	영향력 있는 노드 찾기
커뮤니티 탐지	Louvain, Leiden, Label Propagation	그룹/클러스터 발견
경로 탐색	Shortest Path, All Pairs Shortest Path	최단 경로 계산
유사도	Node Similarity, Cosine Similarity	유사한 노드 찾기
임베딩	FastRP, GraphSAGE	노드 벡터화

GDS 사용 방법

1. Python 라이브러리 설치

```
pip install graphdatascience
```

2. 기본 사용 흐름

1. **그래프 프로젝션**: Neo4j 데이터를 GDS 형식으로 변환
2. **알고리즘 실행**: 원하는 알고리즘 선택 및 실행
3. **결과 활용**: 결과를 노드 속성으로 저장하거나 분석

그래프 프로젝션(Graph Projection)

그래프 프로젝션이란?

- **Neo4j의 실제 데이터를 메모리 내 그래프 구조로 변환**하는 과정입니다.
- GDS 알고리즘을 실행하기 전에 반드시 필요한 단계입니다.
- Neo4j 데이터베이스의 노드와 관계를 GDS가 효율적으로 처리할 수 있는 형식으로 변환합니다.

왜 그래프 프로젝션이 필요한가?

1. **성능 향상**: 메모리 내 그래프 구조로 변환하여 알고리즘 실행 속도가 크게 향상됩니다.
2. **선택적 로딩**: 필요한 노드, 관계, 속성만 선택하여 프로젝션할 수 있습니다.
3. **최적화**: GDS가 그래프 구조를 최적화하여 대규모 데이터도 효율적으로 처리합니다.

그래프 프로젝션 방법

기본 프로젝션

```
G, result = gds.graph.project(
    "graph-name",          # 그래프 이름
    ["NodeLabel"],         # 노드 레이블 리스트
    "RELATIONSHIP_TYPE"    # 관계 타입
)
```

속성 포함 프로젝션

```
G, result = gds.graph.project(
    "graph-name",
    ["Movie", "User"],                # 여러 노드 레이블
    "RATED",                          # 관계 타입
    nodeProperties=["title", "voteAverage"],      # 노드 속성
    relationshipProperties=["rating"]             # 관계 속성
)
```

프로젝션된 그래프 관리

그래프 목록 확인

```
graphs = gds.graph.list()
```

그래프 정보 확인

```
print(f"노드 수: {G.node_count()}")
print(f"관계 수: {G.relationship_count()}")
```

그래프 삭제

```
gds.graph.drop("graph-name")
```

프로젝션 전략

- **목적별 프로젝션**: 분석 목적에 따라 필요한 노드와 관계만 선택
- **메모리 관리**: 사용하지 않는 그래프는 삭제하여 메모리 절약
- **재사용**: 동일한 그래프 구조를 여러 알고리즘에 재사용 가능

GDS vs 일반 그래프 분석

구분	일반 Cypher 쿼리	GDS
목적	데이터 조회 및 탐색	그래프 분석 및 ML
성능	중소규모 데이터	대규모 데이터 최적화
알고리즘	기본 경로 탐색	전문 알고리즘 제공
사용 예	`MATCH (a)-[:KNOWS]->(b)`	PageRank, Community Detection

활용 분야

분야	활용 예
추천 시스템	PageRank로 영향력 있는 제품/사용자 찾기
소셜 네트워크 분석	커뮤니티 탐지로 그룹 구조 파악
사기 탐지	이상 패턴 탐지 및 네트워크 분석
GraphRAG	노드 임베딩을 통한 의미 기반 검색
지식 그래프	엔티티 간 중요도 및 유사도 계산

GDS 설치 및 설정

Neo4j에서 GDS 활성화

- **Neo4j Desktop**: 플러그인으로 설치
- **Neo4j Aura**: Enterprise 버전에서 자동 제공
- **Docker**: GDS 플러그인 포함 이미지 사용

Python 라이브러리

```
from graphdatascience import GraphDataScience
```

정리

- GDS는 Neo4j의 그래프 데이터 과학 라이브러리
- 다양한 그래프 알고리즘을 제공하여 복잡한 분석 가능
- Python `graphdatascience` 라이브러리로 쉽게 사용 가능
- 추천 시스템, 네트워크 분석, GraphRAG 등 다양한 분야에 활용

다음 단계

- **실습 예제**: `4. GDS(Graph Data Science) 예제.ipynb` 에서 자세한 사용법 학습
- **공식 문서**: Neo4j GDS Documentation
- **알고리즘 가이드**: 각 알고리즘의 상세 설명 및 파라미터 튜닝